Portfolio và phỏng vấn DevOps

Một chủ đề, một tài liệu: mô hình, bài tập, lỗi thường gặp và câu hỏi phỏng vấn.

Đọc theo thứ tự trong tập. Làm lab trên môi trường riêng, ghi kết quả và đối chiếu tài liệu chính thức theo phiên bản bạn đang dùng.

Bám theo bốn JD người học cung cấp | 11/10/2026

Tài liệu độc lập theo một chủ đề. Đọc mô hình trước, làm lab trên môi trường riêng, ghi bằng chứng và tự giải thích failure mode. Các ví dụ dựa trên ứng dụng Notes API và bốn JD đã cung cấp.

Chọn nhánh chuyên sâu có chủ đích

Sau các giai đoạn chung, chọn một hoặc hai nhánh. DevOps/Cloud đào sâu IAM, network, multi-environment, IaC và governance. SRE đào sâu SLI/SLO, alerting, incident command, capacity và giảm công việc thủ công lặp lại. Platform đào sâu trải nghiệm của nhóm phát triển, self-service, tiêu chuẩn triển khai và cách đo mức chấp nhận của nền tảng. Delivery/DevSecOps đào sâu test strategy, release safety, quyền pipeline, provenance và auditability.

Một nhánh không xóa kiến thức chung. SRE vẫn cần hiểu deploy; Platform vẫn cần hiểu security; Cloud vẫn cần biết ứng dụng hoạt động ra sao. Hãy chọn theo vấn đề muốn giải và vị trí tuyển dụng thực tế, rồi lấy một hệ thống cụ thể để đi sâu. Nếu không chắc, giữ DevOps Engineer làm nhánh chính và bổ sung SRE ở mức vận hành.

GitOps dùng Git làm nguồn mô tả trạng thái và controller đồng bộ môi trường; progressive delivery kiểm soát phạm vi người dùng khi phát hành; policy as code kiểm tra quy tắc bằng mã. Đây là công cụ/phương pháp có ích khi số môi trường, nhóm hoặc yêu cầu kiểm soát đủ lớn. Trước khi thêm vào portfolio, viết vấn đề hiện tại mà chúng giải quyết và chi phí vận hành mới.

Thiết kế dự án portfolio xuyên suốt

Đề bài đề xuất: API ghi chú hoặc một dịch vụ web nhỏ có database. Người dùng có thể tạo và đọc dữ liệu; repository có test; CI tạo artifact; ứng dụng chạy trong container; một môi trường local và một môi trường triển khai thử nghiệm có hướng dẫn; IaC mô tả tài nguyên cloud nếu dùng cloud; logs/metrics cho phép điều tra; backup/restore và rollback được diễn tập. Nếu chưa có ngân sách, phần cloud có thể ở dạng thiết kế và lab nhỏ có cleanup rõ; đừng viết trong README rằng hạ tầng đã chạy nếu bạn chưa thực sự chạy.

Chia dự án thành các mốc review được. Mốc 1: app và test chạy local, README nói cách chạy. Mốc 2: Docker/Compose chạy app và database, dữ liệu sống qua restart. Mốc 3: CI kiểm tra và build artifact có version. Mốc 4: deploy, cấu hình và quyền tối thiểu. Mốc 5: dashboard, alert và runbook. Mốc 6: diễn tập lỗi, rollback, backup/restore và viết postmortem. Mỗi mốc có commit, bằng chứng và một quyết định kỹ thuật ngắn.

Cấu trúc README hữu ích

README nên trả lời: dự án giải quyết gì; kiến trúc bằng sơ đồ; cách chạy local; cách test; cách deploy/rollback; secrets được cung cấp ra sao mà không lộ giá trị; cách xem log/metrics; chi phí dự kiến và cleanup; giới hạn hiện tại. Dùng ảnh minh họa có chú thích nhưng đừng biến ảnh thành bằng chứng duy nhất: người khác cần có thể lặp lại bằng lệnh/hướng dẫn. Nếu chưa làm xong một phần, ghi "chưa triển khai" và kế hoạch thay vì mô tả như đã hoàn thành.

Bài tập review như người phỏng vấn

Tự mở repo trên một máy hoặc tài khoản mới. Chỉ đọc README, bạn có thể chạy được không? Lệnh nào thất bại? File nào chứa cấu hình mặc định? Nếu xóa một container, data có còn không? Nếu phiên bản mới lỗi, mất bao lâu để quay về? Ghi các câu trả lời cùng kết quả quan sát được và sửa README sau mỗi lần thử.

Cách trả lời phỏng vấn kỹ thuật

Một câu trả lời mạnh có cấu trúc: xác định mục tiêu và tác động -> nêu giả thuyết -> tìm bằng chứng -> thay đổi nhỏ có rollback -> xác minh -> phòng ngừa lặp lại. Ví dụ câu hỏi "ứng dụng báo 502 sau deploy": kiểm tra thời điểm bắt đầu, phạm vi ảnh hưởng và thay đổi gần nhất; xem proxy/upstream, endpoints, readiness và log; nếu liên quan release mới, cân nhắc rollback để phục hồi; sau đó tìm nguyên nhân gốc. Bạn không cần đoán một nguyên nhân duy nhất từ triệu chứng mơ hồ.

Khi hỏi về công cụ, giải thích bài toán trước cú pháp. "Vì sao dùng Terraform?" có thể trả lời: để mô tả và review thay đổi hạ tầng, tái tạo môi trường, quản lý drift; đồng thời phải bảo vệ state và kiểm tra plan. "Vì sao dùng Kubernetes?" cần nêu nhu cầu điều phối workload và chi phí vận hành; không phải mọi ứng dụng cần cluster.

Phỏng vấn hành vi thường hỏi lần bạn xử lý lỗi, làm việc với nhóm hoặc cân bằng tốc độ và an toàn. Dùng cấu trúc STAR: tình huống, nhiệm vụ, hành động, kết quả. Nếu ví dụ là lab cá nhân, nói rõ đó là lab; không nhận là sự cố sản xuất. Kết quả nên đo được: thời gian phát hiện/khôi phục trong diễn tập, số bước thủ công được giảm, tỷ lệ test qua hoặc tài liệu giúp người khác tái tạo.

Bộ câu hỏi tự luyện

Linux: service không khởi động, bạn kiểm tra theo thứ tự nào? Networking: DNS đúng nhưng kết nối timeout, bạn thu thập bằng chứng gì? Git/CI: test pass local nhưng fail runner, khác biệt môi trường nào cần xem? Docker: container restart làm mất dữ liệu, vì sao? Cloud: một VM public truy cập được nhưng database không, các lớp quyền/mạng nào cần kiểm tra? Terraform: plan thay database, bạn quyết định ra sao? Kubernetes: Pod Running nhưng Service không có endpoint, tại sao? SRE: một alert liên tục báo CPU 90% nhưng người dùng không bị ảnh hưởng, bạn chỉnh cách cảnh báo ra sao?

Đối với mỗi câu, tự trả lời thành tiếng trong hai phút, rồi bổ sung một ví dụ từ chính repo của mình. Nếu chưa từng làm, tạo lab nhỏ an toàn để lấy bằng chứng. Không học thuộc một "đáp án vàng" vì cấu hình công ty khác nhau.

Phân biệt sẵn sàng ứng tuyển và trình độ cao

Một portfolio có thể chứng minh bạn có nền tảng và khả năng học. Trình độ cao đòi hỏi quyết định khi yêu cầu mâu thuẫn, sự cố không rõ nguyên nhân, nhiều nhóm cùng phụ thuộc, chi phí thật và thay đổi có hậu quả. Những trải nghiệm này tích lũy qua công việc, dự án mở, cộng tác và diễn tập gần với thực tế. Mục tiêu ngắn hạn của khóa học là giúp bạn giải thích đúng, làm được lab và có bằng chứng để bắt đầu nhận trách nhiệm phù hợp.

Hãy đánh giá tiến bộ bằng năng lực: bạn có thể triển khai lại từ README, lần theo lỗi từ triệu chứng, khôi phục dữ liệu, giải thích chi phí/bảo mật và nêu khi nào nên chọn giải pháp đơn giản hơn không? Số

chứng chỉ và tên công cụ trong CV chỉ là phụ. Nếu còn lỗ hổng, quay lại giai đoạn liên quan, làm một lab nhỏ rồi cập nhật portfolio.

Hồ sơ bằng chứng cho từng năng lực

Một portfolio đáng tin cho người mới nên có bảng năng lực -> bằng chứng. Ví dụ: Git -> lịch sử commit và pull request có mô tả; CI -> workflow và một run thất bại được sửa; Docker -> Dockerfile, Compose và ảnh/log cho dữ liệu sống sau restart; IaC -> file Terraform và bản plan đã xem xét; vận hành -> dashboard, runbook, postmortem lab; bảo mật -> threat model, quyền pipeline và cách quản lý secret. Không cần che giấu lỗi từng gặp: một lỗi được điều tra có phương pháp là bằng chứng học tốt.

Mỗi mục bằng chứng phải trả lời: bạn tự làm phần nào, làm trong môi trường nào, người khác lặp lại ra sao và giới hạn gì. Không dùng từ "production-ready" nếu chưa có test tải, bảo mật, backup, vận hành và bối cảnh yêu cầu. Việc nói rõ "lab local" và "tài nguyên cloud thử nghiệm đã xóa" làm báo cáo chính xác hơn.

Rubric tự chấm 0-2

Chấm mỗi mục: 0 = chưa có, 1 = có nhưng cần hướng dẫn nhiều, 2 = tự làm và giải thích được. Các mục gồm: chạy/test app; Git/CI; Docker/data persistence; cloud/IAM/network; Terraform plan/state; deploy/rollback; logs/metrics/SLO; backup/restore; security review; README tái tạo. Tổng điểm không phải chứng nhận trình độ; nó chỉ giúp chọn bài cần ôn. Với mục 1 điểm, viết rõ bước nào mình bị kẹt; đó là đầu vào cho buổi học tiếp theo.

Hai tình huống phỏng vấn có đáp án mẫu

Tình huống 1: "Pipeline xanh, deploy xong người dùng gặp 502." Câu trả lời mẫu: đầu tiên xác nhận phạm vi/tác động và thời điểm; kiểm tra artifact/version vừa deploy; xem proxy hoặc ingress có upstream/endpoint Ready không; đọc log API và health; nếu có bằng chứng bản mới gây lỗi, rollback theo kế hoạch; xác nhận request trở lại thành công; sau đó tìm vì sao test/smoke chưa bắt được. Các nhánh điều tra cụ thể tùy bằng chứng: selector sai, port sai, readiness fail, app crash hoặc kết nối database lỗi. Đừng khẳng định một nguyên nhân khi câu hỏi chưa cung cấp log.

Tình huống 2: "Terraform plan muốn replace database production." Câu trả lời mẫu: không apply; đọc thuộc tính gây replacement và thay đổi từ commit nào; xác định dữ liệu, backup/restore, RPO/RTO và tác động; hỏi có cách thay đổi ít rủi ro hơn hoặc migration tương thích không; review với người sở hữu dữ liệu và quy trình thay đổi; chỉ thực hiện sau khi có kế hoạch, thời điểm và rollback phù hợp. Điều này cho thấy bạn hiểu Terraform chỉ mô tả thao tác, còn trách nhiệm đánh giá hậu quả thuộc nhóm vận hành.

Bài tập trả lời của chính bạn

Ghi âm hoặc viết câu trả lời 2 phút cho mỗi tình huống trước khi xem mẫu. Sau đó so sánh: bạn có xác định tác động, bằng chứng, hành động giảm ảnh hưởng, xác minh và phòng ngừa chưa? Lặp lại sau một tuần mà không nhìn tài liệu. Nếu không thể giải thích một bước từ chính lab, quay lại làm lab; học thuộc câu mẫu không đủ.

Bản ghi quyết định kỹ thuật và giới hạn dự án

Viết một ADR ngắn cho một lựa chọn thật của dự án. Cấu trúc: bối cảnh (API ghi chú dùng cho ai, traffic, dữ liệu), quyết định (ví dụ SQLite trong lab local), lý do (không cần server DB, dễ học dữ liệu bền), lựa chọn khác (PostgreSQL), hệ quả (khó phù hợp nhiều tiến trình/máy, backup và truy cập khác), điều kiện xem lại (cần nhiều người dùng/đồng thời hoặc cloud production). Một ADR tốt không cần dài, nhưng phải đủ để người khác hiểu tại sao bạn chọn như vậy ở thời điểm đó.

Làm thêm ADR cho câu hỏi "có cần Kubernetes không?". Với dự án nhỏ, bạn có thể dùng Kubernetes để **học**, đồng thời ghi rõ giải pháp vận hành đơn giản hơn có thể phù hợp cho hệ thống thật. Người phỏng vấn sẽ thấy bạn phân biệt mục tiêu học công cụ với quyết định kiến trúc kinh doanh.

Kế hoạch phát triển sau portfolio đầu tiên

Sau dự án đầu, đừng nhân bản thêm ba project giống hệt. Chọn ràng buộc mới để buộc mình học sâu: dữ liệu cần phục hồi trong thời gian ngắn hơn; nhiều môi trường; lưu lượng tăng; nhiều service; nhóm khác cần tự triển khai; yêu cầu kiểm toán quyền. Chọn một nhánh chuyên sâu và đo kết quả. Ví dụ nhánh SRE có thể cải thiện alert để giảm nhiễu và diễn tập failover; nhánh Platform có thể xây một template deployment tự phục vụ cho hai ứng dụng khác nhau.

Tìm feedback từ người khác bằng yêu cầu cụ thể: họ có thể chạy repo theo README không, hiểu rủi ro lớn nhất không, có thấy bước nào chứa secret hoặc có thể tốn phí không, runbook có đủ để xử lý lỗi lab không. Ghi feedback và sửa tài liệu/mã. Vòng lặp nhận phản hồi và cải thiện gần với công việc DevOps hơn việc chỉ sưu tập công cụ.

Portfolio phải chứng minh quyết định và vận hành

Một ảnh “Argo CD xanh” không chứng minh bạn hiểu DevOps. Dự án mạnh cho người phỏng vấn thấy: yêu cầu, kiến trúc, vì sao chọn công cụ, source và IaC, đường build/release, kiểm thử, telemetry, bảo mật, lỗi bạn đã gây có kiểm soát, cách bạn tìm và sửa, giới hạn còn lại. Nếu không có cloud budget, local kind/VM vẫn có giá trị khi bạn nói rõ điều gì chưa mô phỏng được: multi-zone HA, managed IAM, storage enterprise, traffic thật và on-call production. Đừng viết “kinh nghiệm production 3 năm” từ một lab.

Chọn một sản phẩm nhỏ xuyên suốt, ví dụ Notes API. Version 1 chạy trên Linux VM với systemd/Nginx/SQLite để học process, cổng và DB. Version 2 đóng Docker và Compose, thêm test/pipeline. Version 3 triển khai Kubernetes/Helm/Argo, metrics/logs, secret, backup/restore. Version 4 là thiết kế hybrid hoặc Azure theo JD, có IaC và cost/DR plan. Mỗi version giải một vấn đề mới; không thêm công cụ chỉ để tăng danh sách CV. Mục tiêu là kể được một câu chuyện kỹ thuật nối các lớp.

Hồ sơ bằng chứng cho một dự án

Repository nên có README chạy local, sơ đồ luồng request và delivery, cấu trúc thư mục rõ, script/lab dùng được, pipeline config, IaC/manifest, runbook, bảng test/smoke và phần cleanup. Tạo **docs/decisions/** cho ADR ngắn: bối cảnh, lựa chọn, tradeoff, điều kiện xem lại. Ví dụ “chọn SQLite cho lab một instance vì đơn giản; khi scale API nhiều replica chuyển sang DB server managed/on-prem có backup/replication”; đây là đánh giá trưởng thành hơn khoe “đã cài MySQL”.

Evidence report nên ghi ngày, phiên bản tool, commit/digest, lệnh đã chạy, **kết quả thực đã quan sát**, lỗi gặp, cách sửa và điều gì chưa thử. Khi chụp output, xóa token, IP nội bộ nhạy cảm và dữ liệu cá nhân. Nếu không đủ tài nguyên chạy Ceph/ECK, hãy viết mô hình, bài toán capacity, rủi ro và kế hoạch test thực ở môi trường doanh nghiệp; không khẳng định đã vận hành chúng.

Bài tập mock interview theo JD

Câu 1: “Pipeline xanh nhưng sau release tỷ lệ 5xx tăng, bạn làm gì?” Trả lời theo thứ tự: xác định tác động/đặt incident, mốc release/digest, metric theo route, log/trace request lỗi, diff config, Pod/Service/DB, quyết định giảm tác động và rollback có điều kiện, xác nhận phục hồi, postmortem. Không liệt kê 20 lệnh thiếu giả thuyết.

Câu 2: “Ba VM K8s ở VMware, mỗi VM khác host, đủ HA chưa?” Hỏi tiếp về rack/power/datastore/network/LB/DB, capacity N-1, etcd/control plane, storage, RTO/RPO và thử failover. Nếu chưa có thông tin, nêu giả định và giới hạn, không trả lời “đủ” chỉ vì ba host.

Câu 3: “Bạn biết Terraform đến mức nào?” Đưa một plan có thay đổi tạo/thay/xóa, giải thích state/backend/lock, drift, review, permission, destroy/restore và một ví dụ bạn tự làm. Nếu mới lab local, nói rõ và mô tả cách bạn sẽ áp dụng trên cloud dưới review. Trung thực về độ sâu giúp người phỏng vấn đặt câu hỏi đúng.

Kế hoạch ôn phỏng vấn có bằng chứng

Lập ma trận JD: mỗi dòng là một trách nhiệm, ví dụ “triển khai GitLab CI”, “xử lý MySQL replication lag”, “tối ưu Nginx 502”, “Azure Private Endpoint”. Cột tiếp theo: khái niệm giải thích được, lab đã làm, lỗi đã xử lý, bằng chứng URL/file, giới hạn. Điền “chưa làm” thay vì dấu tick mơ hồ. Ưu tiên khoảng trống xuất hiện ở nhiều JD: Linux, networking, Git, Docker, CI/CD, IaC, K8s, observability, security, database. Công cụ chuyên biệt như WSO2/Oracle/Ceph có thể học theo vai trò mục tiêu sau nền tảng.

Mỗi tuần chọn một năng lực, thực hiện ba vòng: **giải thích** (viết sơ đồ 1 trang), **làm** (lab có bằng chứng), **phá và sửa** (failure injection có kiểm soát). Sau đó nhờ một người hoặc tự đóng vai interviewer hỏi “vì sao?” ba lần. Nếu câu trả lời cuối chỉ là “vì tutorial bảo vậy”, quay lại mô hình. Lặp lại lab sau một tuần mà không nhìn tài liệu để kiểm tra bạn còn nhớ cách suy luận, không chỉ nhớ lệnh.

Mẫu câu trả lời trung thực và sắc nét

“Trong dự án lab của tôi, tôi triển khai Notes API trên kind bằng Helm và Argo CD. Tôi cố tình sửa sai Service selector; Pod vẫn Ready nhưng EndpointSlice không có backend, nên request qua Service lỗi. Tôi so labels/selector, khôi phục manifest trong Git, sync lại và xác nhận HTTP 200. Điều tôi chưa thử là cluster nhiều zone và ingress/LB production; nếu áp dụng thật, tôi sẽ thêm topology, SLO, capacity và rollback plan.” Câu trả lời có **bối cảnh, hành động, bằng chứng, kết quả và giới hạn**; tốt hơn một danh sách tên công cụ.

Câu hỏi có lời giải: chứng chỉ có thay portfolio không? Không; chứng chỉ chứng minh một loại kiến thức ở thời điểm thi, portfolio chứng minh cách làm và suy luận; cả hai không tự thay kinh nghiệm vận hành thật. **Một dự án gồm 15 tool có mạnh hơn ba dự án nhỏ?** Chỉ khi các tool có lý do, tích hợp đúng và có bằng chứng; độ rộng không bù lỗi nền tảng. Nguồn đọc thêm: DORA capabilities và Google SRE Workbook.

Rubric tự chấm trước khi đưa dự án vào CV

Chấm mỗi năng lực theo 0-3: **0** chưa làm; **1** đọc và giải thích được; **2** lab chạy có bằng chứng, gây/sửa một lỗi; **3** thiết kế tradeoff, automation, metric/rollback và có người khác review hoặc dùng được. Không tự cho điểm 3 vì xem tutorial. Đối với Linux, ví dụ điểm 2 là tự chẩn đoán process/socket/quyền/đĩa trên VM lab; điểm 3 đòi runbook/automation có kiểm chứng và đánh giá giới hạn. Với Kubernetes, điểm 2 gồm Deployment/Service/probe/EndpointSlice/rollout và một failure injection; điểm 3 gồm topology/capacity, security, observability và release plan. Điểm 3 lab vẫn chưa bằng nhiều năm on-call production; ghi rõ bối cảnh.

Checklist trình bày một case 5 phút

- Bài toán và ràng buộc: số dịch vụ, traffic, RTO/RPO, ngân sách lab. 2. Kiến trúc và vì sao chọn công cụ, kể cả phương án đơn giản hơn. 3. Một thay đổi bạn đã làm, commit/digest và gate. 4. Một sự cố bạn tự tạo hoặc gặp thật: triệu chứng, giả thuyết, bằng chứng, sửa và xác nhận. 5. Điều chưa làm và kế hoạch nếu chuyển production. Tập kể bằng sơ đồ một trang; người phỏng vấn có thể đào bất kỳ mũi tên nào, nên chỉ đưa thứ bạn giải thích được.

Bài tập: viết hai dòng CV. Dòng yếu: “Experienced Kubernetes, Jenkins, Terraform, ELK, Azure.” Dòng mạnh phải có hành động/đầu ra/bằng chứng đúng sự thật, ví dụ “Xây pipeline cho Notes API: test + build image digest + triển khai staging bằng Argo CD; mô phỏng lỗi readiness và ghi runbook xác nhận phục hồi.” Không bịa số uptime hay tiết kiệm chi phí. Nếu chưa làm thực tế, dùng từ “dự án lab” rõ ràng. Nguồn: DORA capabilities.

Tự đánh giá

- Giải thích mô hình bằng lời của bạn, không dựa vào tên công cụ.
- Làm lab và lưu kết quả mong đợi cùng kết quả thực tế.
- Tạo một lỗi có kiểm soát, dùng bằng chứng để khoanh vùng và phục hồi.
- Trả lời một câu hỏi thiết kế: tại sao chọn phương án này, tradeoff và giới hạn là gì?

Nguồn chính thức và đọc thêm

- https://dora.dev/capabilities/
- https://sre.google/workbook/